

Universidad de las Fuerzas Armadas ESPE

Departamento: Ciencias de la Computación

Carrera: Ingeniería de Software

Asignatura: Análisis y Diseño de Software

Instructora del curso: Ing. Jenny A Ruiz R MIS

Apellidos y nombres de los estudiantes:

- Bryan Alexander Rosero Delgado
- Cámeron Victoria Díaz Caisaluiza
- Benjamin Alejandro Robalino

NRC: 36839

Fecha de realización: 22/10/2026

U1 Guia2

GUÍA 2 PARA DESARROLLAR EN PLANTUML

Relación entre casos de uso, diagrama de clases de análisis y diagramas de secuencia de análisis

Caso base: CRUD de Canchas

Elemento	Descripción
Asignatura	Análisis y Diseño de Software
Tema	Trazabilidad entre casos de uso, clases de análisis y secuencias de análisis con PlantUML
Caso base	SPR – Sistema de Pedidos y Reservas de Canchas Don Pepito, con los datos de reserva, pedido, pago y reporte de ingresos
Operaciones CRUD	Realizar Reserva, Realizar Pedido, Realizar Pago y Generar Reporte de Ingresos
Referencia base	Guía 1: diagramas de casos de uso de análisis del SPR (versión final)
Propósito	Pasar del análisis funcional al análisis estructural y dinámico del SPR, manteniendo coherencia entre requisito, caso de uso, clase y secuencia.

1. Propósito de la guía

Esta guía complementa la guía inicial de casos de uso. Su finalidad es analizar el SPR de Canchas Don Pepito de forma progresiva mediante tres modelos conectados: casos de uso de análisis, clases de análisis y diagramas de secuencia de análisis.

El énfasis no está en programar la interfaz, sino en comprender qué responsabilidad cumple cada elemento del modelo y cómo se conserva la trazabilidad desde el requisito funcional del SRS hasta la interacción entre objetos.

2. Relación general entre los tres modelos

Modelo	Pregunta que responde	Producto en PlantUML	Uso en la siguiente etapa
Casos de uso de análisis	¿Qué necesita hacer el actor con el sistema?	Diagrama de casos de uso y matriz RF-CU.	Permite identificar responsabilidades, clases frontera, control y entidad.
Clases de análisis	¿Qué objetos participan para cumplir los casos de uso?	Diagrama de clases de análisis con estereotipos boundary, control, entity y repositorio.	Define los objetos que aparecerán en los diagramas de secuencia.
Secuencias de análisis	¿Cómo colaboran actor, frontera, control y entidad para ejecutar cada caso de uso?	Diagramas de secuencia por operación del CRUD del estudiante	Valida que el flujo sea coherente con los casos de uso y las clases identificadas.

3. Caso base actualizado: SPR - Canchas Don Pepito

El caso se basa en una interfaz sencilla denominada SPR - Canchas Don Pepito, con 10 campos de entrada y 8 operaciones principales.

Dato	Tipo sugerido	Descripción	Validación mínima
ID de reserva	Entero	Identificador único de la reserva.	Obligatorio y no repetido.
Cancha	Texto / referencia	Cancha reservada por el cliente.	Obligatoria; debe estar habilitada.
Fecha y horario	Fecha y hora	Franja de 1 hora de la reserva.	Dentro del horario de atención, posterior al momento actual, máximo 30 días de anticipación y sin solapamiento.
Estado de la reserva	Texto	Estado actual de la reserva.	«Pendiente» al realizarse; «confirmada» cuando el administrador la acepta.
ID de pedido	Entero	Identificador único del pedido de productos.	Obligatorio y no repetido.
Producto y cantidad	Referencia y entero	Productos solicitados y su cantidad.	El 100% de los productos debe tener stock suficiente.
Ubicación de entrega	Coordenadas GPS o zona	Lugar donde se entrega el pedido.	GPS válido con consentimiento, o zona seleccionada manualmente.
Método de pago	Texto	Efectivo o transferencia.	Obligatorio; se selecciona en un solo paso.
Monto	Decimal	Valor a cobrar del pedido o reserva.	Numérico, con hasta 2 decimales.
Estado del pago	Texto	Pendiente, confirmado, rechazado o reembolsado.	El pago expira si no se confirma en 15 minutos.

Botón / Acción de interfaz	Caso de uso asociado	Descripción de análisis
Reservar	Realizar Reserva	Registra la solicitud de reserva de una cancha en estado pendiente.
Aceptar reserva	Aceptar Reserva	El administrador confirma la reserva; se registra y se bloquea el horario.
Realizar pedido	Realizar Pedido	Registra un pedido de productos validando el stock disponible.
Alerta de pedido	Notificar Pedido	Avisa al vendedor, con alerta visual y sonora, que hay un pedido nuevo.
Realizar pago	Realizar Pago	Registra el pago de un pedido o reserva según el método elegido.
Pagar en efectivo	Pagar Efectivo	El vendedor registra el monto recibido en efectivo.
Pagar por transferencia	Pagar Transferencia	La banca móvil confirma o rechaza la transferencia.
Generar reporte	Generar Reporte de Ingresos	Calcula y presenta los ingresos del periodo con los filtros elegidos.

4. Requisitos funcionales de nivel 0

Código	Requisito funcional de nivel 0	Actor principal	Caso de uso asociado
RF-022	El sistema debe permitir al cliente realizar una reserva indicando cancha, fecha y horario, con duración fija de 1 hora por franja.	Cliente	Realizar Reserva (GRC-27_1)
RF-023	El sistema debe validar que la franja no se solape con otra reserva y bloquear el horario al registrarla.	Administrador	Aceptar Reserva (GRC-27_2)
RF-030	El sistema debe permitir al cliente realizar pedidos de productos, reservando temporalmente el stock solicitado.	Cliente	Realizar Pedido (GT-03)

RF-032	El sistema debe notificar al vendedor, con alerta visual y sonora, la existencia de un nuevo pedido.	Vendedor	Notificar Pedido (GT-04)
RF-038	El sistema debe permitir al cliente iniciar el pago de un pedido o reserva, seleccionando el método de pago.	Cliente	Realizar Pago (GI-33)
RF-039	El sistema debe permitir al vendedor registrar el cobro en efectivo.	Vendedor	Pagar Efectivo (GI-34)
RF-040	El sistema debe procesar el pago por transferencia mediante la plataforma de banca móvil.	Banca Móvil	Pagar Transferencia (GI-35)
RF-044	El sistema debe permitir al administrador generar el reporte de ingresos, filtrable por fechas y método de pago.	Administrador	Generar Reporte de Ingresos (GI-32)

5. Diagrama de casos de uso de análisis

El diagrama de casos de uso representa las funciones visibles para los actores. El Cliente reserva, pide y paga; el Vendedor recibe las alertas de pedidos y registra el cobro en efectivo; el Administrador acepta reservas y genera el reporte; la Banca Móvil confirma las transferencias. Se usa <<extend>> para los comportamientos condicionados, como en el documento de casos de uso del proyecto.

Código PlantUML

```
@startuml
left to right direction
skinparam packageStyle rectangle
skinparam shadowing false

actor "Cliente" as Cliente
actor "Vendedor" as Vendedor
actor "Administrador" as Admin
actor "Banca Móvil" as Banca

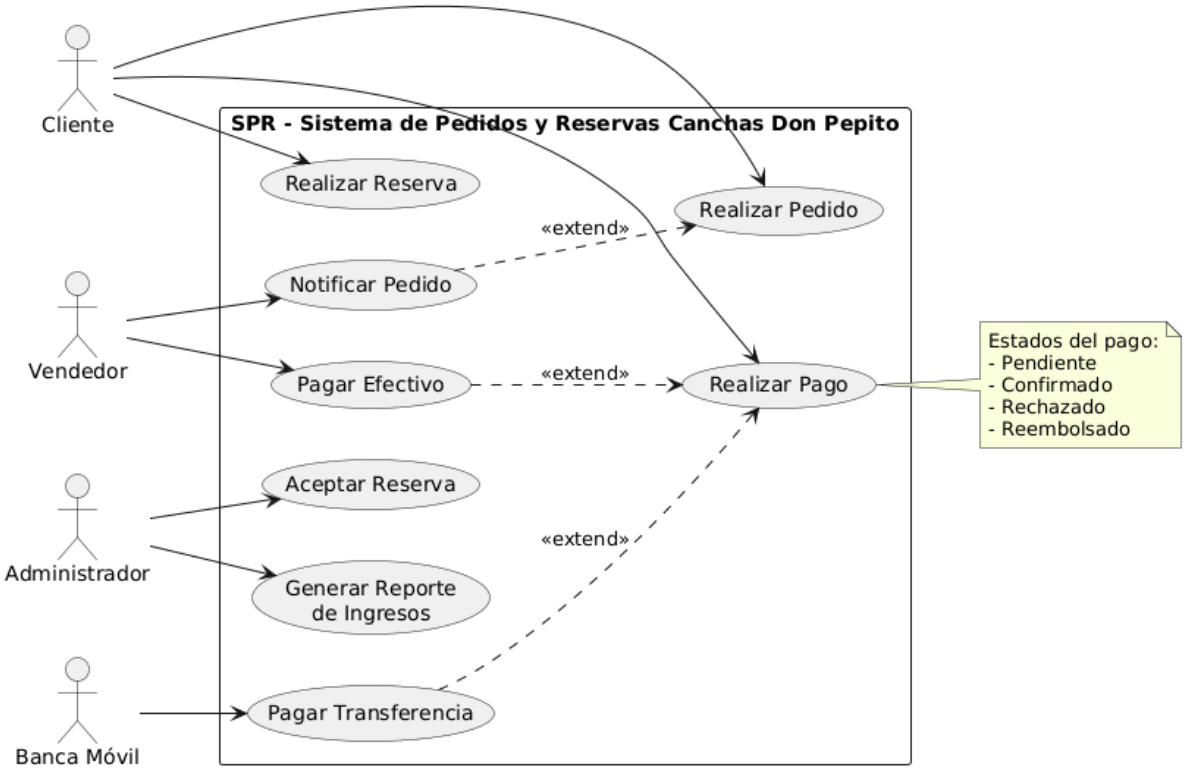
rectangle "SPR - Sistema de Pedidos y Reservas Canchas Don Pepito" {
  usecase "Realizar Reserva" as UC1
  usecase "Aceptar Reserva" as UC2
  usecase "Realizar Pedido" as UC3
  usecase "Notificar Pedido" as UC4
  usecase "Realizar Pago" as UC5
  usecase "Pagar Efectivo" as UC6
  usecase "Pagar Transferencia" as UC7
```

```
  usecase "Generar Reporte\nde Ingresos" as UC8
}

Cliente --> UC1
Cliente --> UC3
Cliente --> UC5
Admin --> UC2
Admin --> UC8
Vendedor --> UC4
Vendedor --> UC6
Banca --> UC7

UC4 ..> UC3 : <<extend>>
UC6 ..> UC5 : <<extend>>
UC7 ..> UC5 : <<extend>>

note right of UC5
Estados del pago:
- Pendiente
- Confirmado
- Rechazado
- Reembolsado
end note
@enduml
```



6. Derivación de clases de análisis

A partir de los casos de uso se identifican las clases de análisis con los estereotipos boundary, control y entity. Se incluye además un repositorio por entidad para representar la persistencia sin entrar aún en detalles técnicos de base de datos. La Banca Móvil, al ser un sistema externo, se representa con una clase frontera.

Tipo de clase	Responsabilidad en el análisis	Clase propuesta	Relación con el SPR
Boundary / Frontera	Recibe datos del actor y muestra resultados.	PantallaReserva	Captura cancha, fecha y horario; permite reservar y aceptar reservas.
Boundary / Frontera	Recibe datos del actor y muestra resultados.	PantallaPedido	Captura productos, cantidades y ubicación de entrega.
Boundary / Frontera	Avisa al vendedor de un pedido nuevo.	AlertaPedido	Muestra la alerta visual y sonora con el detalle del pedido.

Boundary / Frontera	Recibe datos del actor y muestra resultados.	PantallaPago	Captura método de pago, monto recibido y comprobante.
Boundary / Frontera	Recibe datos del actor y muestra resultados.	PantallaReporteIngresos	Captura fechas y método de pago; muestra el reporte.
Boundary / Frontera	Comunica el sistema con el actor externo.	InterfazBancaMovil	Consulta a la banca móvil la confirmación de una transferencia.
Control	Coordina la lógica de cada caso de uso.	ControlReserva	Valida disponibilidad, registra la reserva pendiente y la confirma.
Control	Coordina la lógica de cada caso de uso.	ControlPedido	Valida stock, registra el pedido y notifica al vendedor.
Control	Coordina la lógica de cada caso de uso.	ControlPago	Registra el pago y coordina el cobro en efectivo o por transferencia.
Control	Coordina la lógica de cada caso de uso.	ControlReporte	Consulta los pagos confirmados y calcula el total de ingresos.
Entity / Entidad	Representa la información del dominio.	Reserva, Cancha	Contienen los datos de la reserva y de la cancha.
Entity / Entidad	Representa la información del dominio.	Pedido, Producto	Contienen los datos del pedido y del catálogo.
Entity / Entidad	Representa la información del dominio.	Pago	Contiene método, monto y estado del pago.
Repositorio / Colección	Conserva y recupera registros.	RepositorioReserva, RepositorioCancha, RepositorioPedido, RepositorioProducto, RepositorioPago	Permiten guardar, buscar, actualizar y listar cada entidad.

Código PlantUML del diagrama de clases de análisis

```
@startuml
skinparam classAttributeIconSize 0
skinparam shadowing false
left to right direction

class PantallaReserva <<boundary>> {
 - cmbCancha: String
 - txtFecha: String
 - txtHorario: String
 + clickReservar()
 + clickAceptarReserva()
 + mostrarMensaje(mensaje: String)
}

class PantallaPedido <<boundary>> {
 - listaProductos: Map<String, int>
 - ubicacion: String
 + clickRealizarPedido()
 + mostrarMensaje(mensaje: String)
}

class AlertaPedido <<boundary>> {
 + mostrarAlerta(pedido: Pedido)
}

class PantallaPago <<boundary>> {
 - cmbMetodoPago: String
 - txtMontoRecibido: String
 - comprobante: String
 + clickRealizarPago()
 + clickPagarEfectivo()
 + clickPagarTransferencia()
 + mostrarMensaje(mensaje: String)
}

class PantallaReporteIngresos <<boundary>> {
 - txtFechaInicio: String
 - txtFechaFin: String
 - cmbMetodoPago: String
 + clickGenerarReporte()
 + mostrarReporte(pagos: List<Pago>, total: double)
}

class InterfazBancaMovil <<boundary>> {
 + solicitarConfirmacion(idPago: String, monto: double, comprobante: String): boolean
```

```
}

class ControlReserva <<control>> {
  + realizarReserva(idCancha: String, fecha: String, horario: String, idCliente: String): Resultado
  + aceptarReserva(idReserva: String): Resultado
  + validarDisponibilidad(idCancha: String, fecha: String, horario: String): boolean
}

class ControlPedido <<control>> {
  + realizarPedido(idCliente: String, items: Map<String, int>, ubicacion: String): Resultado
  + validarStock(items: Map<String, int>): boolean
  + notificarPedido(pedido: Pedido): void
}

class ControlPago <<control>> {
  + realizarPago(idReferencia: String, metodo: String, monto: double): Resultado
  + pagarEfectivo(idPago: String, montoRecibido: double): Resultado
  + pagarTransferencia(idPago: String, comprobante: String): Resultado
  + validarMonto(monto: double, montoRecibido: double): boolean
}

class ControlReporte <<control>> {
  + generarReporteIngresos(fechaInicio: String, fechaFin: String, metodoPago: String): List<Pago>
  + calcularTotal(pagos: List<Pago>): double
}

class Reserva <<entity>> {
  - idReserva: String
  - idCliente: String
  - idCancha: String
  - fecha: String
  - horario: String
  - estado: String
  + Reserva(idCliente: String, idCancha: String, fecha: String, horario: String, estado: String)
  + confirmar(): void
  + getEstado(): String
}

class Cancha <<entity>> {
  - idCancha: String
  - nombre: String
  - tarifaHora: double
  - estado: String
  + estaHabilitada(): boolean
}

class Pedido <<entity>> {
```

```
 - idPedido: String
 - idCliente: String
 - items: Map<String, int>
 - ubicacion: String
 - estado: String
 + Pedido(idCliente: String, items: Map<String, int>, ubicacion: String, estado: String)
 + getEstado(): String
}

class Producto <<entity>> {
 - idProducto: String
 - nombre: String
 - precio: double
 - stock: int
 + hayStock(cantidad: int): boolean
}

class Pago <<entity>> {
 - idPago: String
 - idReferencia: String
 - metodo: String
 - monto: double
 - estado: String
 + Pago(idReferencia: String, metodo: String, monto: double, estado: String)
 + confirmar(): void
 + rechazar(): void
 + getMonto(): double
}

class RepositorioReserva <<repository>> {
 + existeSolape(idCancha: String, fecha: String, horario: String): boolean
 + guardar(reserva: Reserva): void
 + buscarPorId(idReserva: String): Reserva
 + actualizar(reserva: Reserva): void
}

class RepositorioCancha <<repository>> {
 + buscarPorId(idCancha: String): Cancha
}

class RepositorioPedido <<repository>> {
 + guardar(pedido: Pedido): void
}

class RepositorioProducto <<repository>> {
 + buscarPorId(idProducto: String): Producto
 + reservarStock(idProducto: String, cantidad: int): void
```

```
}

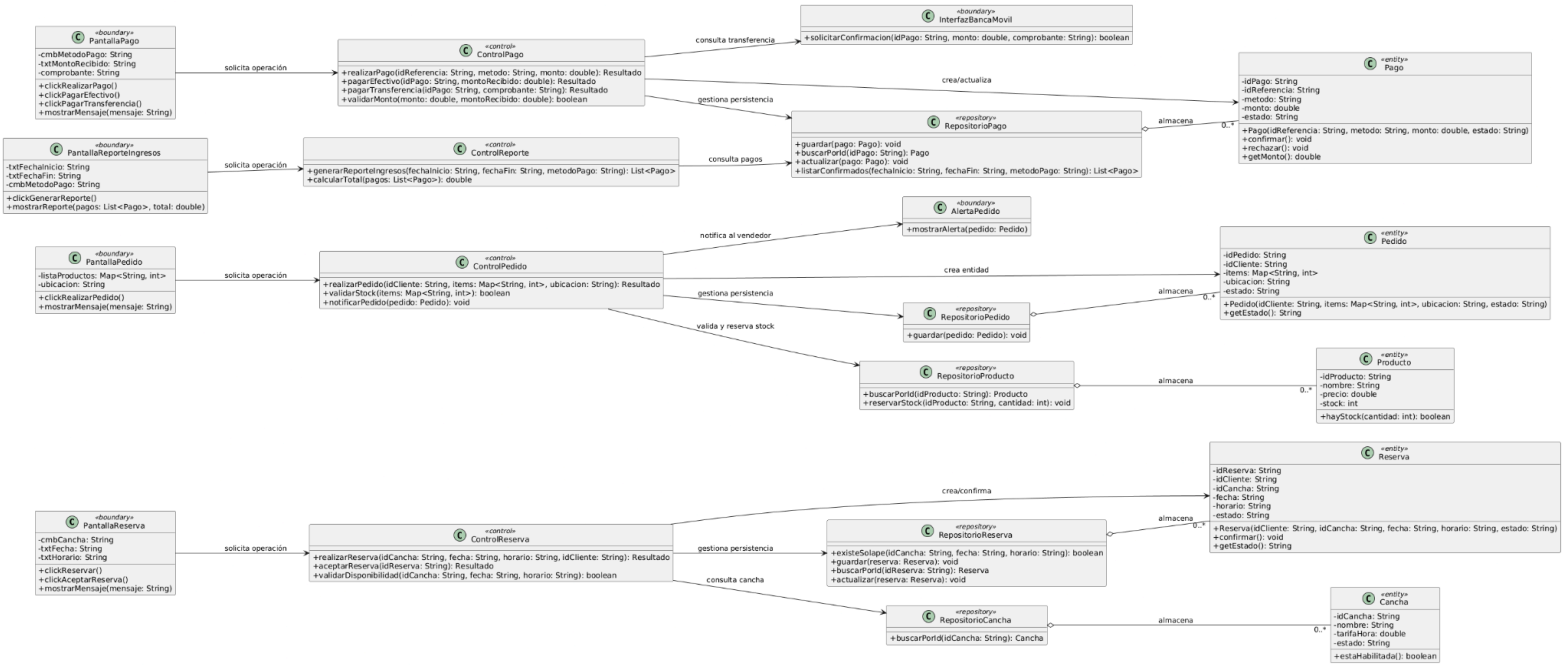
class RepositorioPago <<repository>> {
  + guardar(pago: Pago): void
  + buscarPorId(idPago: String): Pago
  + actualizar(pago: Pago): void
  + listarConfirmados(fechaInicio: String, fechaFin: String, metodoPago: String): List<Pago>
}

PantallaReserva --> ControlReserva : solicita operación
PantallaPedido --> ControlPedido : solicita operación
PantallaPago --> ControlPago : solicita operación
PantallaReporteIngresos --> ControlReporte : solicita operación

ControlPedido --> AlertaPedido : notifica al vendedor
ControlPago --> InterfazBancaMovil : consulta transferencia

ControlReserva --> Reserva : crea/confirma
ControlReserva --> RepositorioReserva : gestiona persistencia
ControlReserva --> RepositorioCancha : consulta cancha
ControlPedido --> Pedido : crea entidad
ControlPedido --> RepositorioPedido : gestiona persistencia
ControlPedido --> RepositorioProducto : valida y reserva stock
ControlPago --> Pago : crea/actualiza
ControlPago --> RepositorioPago : gestiona persistencia
ControlReporte --> RepositorioPago : consulta pagos

RepositorioReserva o-- "0..*" Reserva : almacena
RepositorioCancha o-- "0..*" Cancha : almacena
RepositorioPedido o-- "0..*" Pedido : almacena
RepositorioProducto o-- "0..*" Producto : almacena
RepositorioPago o-- "0..*" Pago : almacena
@enduml
```



7. Matriz de relación: casos de uso, clases y secuencias

Caso de uso	Boundary	Control	Entity / Repositorio	Diagrama de secuencia esperado
Realizar Reserva	PantallaReserva: captura cancha, fecha y horario.	ControlReserva: valida disponibilidad y coordina el registro.	Reserva y Cancha; RepositorioReserva y RepositorioCancha.	Secuencia 8.1: ingresar datos → verificar cancha → validar solapamiento → crear reserva pendiente → guardar → informar.
Aceptar Reserva	PantallaReserva: el administrador acepta la reserva.	ControlReserva: confirma la reserva.	Reserva; RepositorioReserva.	Secuencia 8.1 (segunda fase): aceptar → buscar reserva → confirmar → actualizar → informar.
Realizar Pedido	PantallaPedido: captura productos, cantidades y ubicación.	ControlPedido: valida stock y coordina el registro.	Pedido y Producto; RepositorioPedido y RepositorioProducto.	Secuencia 8.2: ingresar pedido → validar stock → reservar stock → crear pedido pendiente → guardar → notificar.

Notificar Pedido	AlertaPedido: muestra la alerta al vendedor.	ControlPedido: genera la notificación.	Pedido.	Secuencia 8.2 (parte final): notificar pedido → mostrar alerta al vendedor.
Realizar Pago	PantallaPago: captura el método de pago.	ControlPago: registra el pago pendiente.	Pago; RepositorioPago.	Secuencia 8.3: elegir método → crear pago pendiente → guardar → derivar a efectivo o transferencia.
Pagar Efectivo	PantallaPago: el vendedor registra el monto recibido.	ControlPago: valida el monto y confirma.	Pago; RepositorioPago.	Secuencia 8.3 (opción efectivo): registrar monto → validar → confirmar → actualizar.
Pagar Transferencia	PantallaPago e InterfazBancaMovil.	ControlPago: consulta la banca y confirma o rechaza.	Pago; RepositorioPago.	Secuencia 8.3 (opción transferencia): subir comprobante → consultar banca → confirmar o rechazar → actualizar.
Generar Reporte de Ingresos	PantallaReporteIngresos: captura filtros y muestra el reporte.	ControlReporte: consulta pagos y calcula el total.	Pago; RepositorioPago.	Secuencia 8.4: ingresar filtros → listar pagos confirmados → calcular total → mostrar reporte.

8. Diagramas de secuencia de análisis

Los diagramas de secuencia muestran cómo colaboran los objetos de análisis en el tiempo. Se elabora un diagrama por cada caso de uso principal. Los casos de apoyo (Aceptar Reserva, Notificar Pedido, Pagar Efectivo y Pagar Transferencia) quedan dentro del diagrama del caso que los origina.

8.1 Secuencia: Realizar Reserva (incluye Aceptar Reserva)

```
@startuml
title Secuencia de análisis - Realizar Reserva
actor "Cliente" as Cliente
actor "Administrador" as Admin
boundary "PantallaReserva" as UI
control "ControlReserva" as Ctrl
entity "Reserva" as Res
entity "Cancha" as Can
```

```
database "RepositorioCancha" as RepoCan
database "RepositorioReserva" as RepoRes

Cliente -> UI : Selecciona cancha, fecha y horario
Cliente -> UI : Clic en Reservar
UI -> Ctrl : realizarReserva(idCancha, fecha, horario, idCliente)
Ctrl -> RepoCan : buscarPorId(idCancha)
RepoCan --> Ctrl : Cancha encontrada
Ctrl -> Can : estaHabilitada()
Can --> Ctrl : true
Ctrl -> Ctrl : validarDisponibilidad(idCancha, fecha, horario)
Ctrl -> RepoRes : existeSolape(idCancha, fecha, horario)
RepoRes --> Ctrl : false
Ctrl -> Res : new Reserva(idCliente, idCancha, fecha, horario, "pendiente")
Ctrl -> RepoRes : guardar(Res)
RepoRes --> Ctrl : guardado correcto
Ctrl --> UI : Resultado exitoso
UI --> Cliente : Muestra reserva pendiente de confirmar

== Aceptar Reserva ==
Admin -> UI : Selecciona la reserva pendiente
Admin -> UI : Clic en Aceptar reserva
UI -> Ctrl : aceptarReserva(idReserva)
Ctrl -> RepoRes : buscarPorId(idReserva)
RepoRes --> Ctrl : Reserva encontrada
Ctrl -> Res : confirmar()
Ctrl -> RepoRes : actualizar(Res)
RepoRes --> Ctrl : actualización correcta
Ctrl --> UI : Resultado exitoso
UI --> Admin : Muestra reserva confirmada y horario bloqueado
@enduml
```

8.2 Secuencia: Realizar Pedido (incluye Notificar Pedido)

```
@startuml
title Secuencia de análisis - Realizar Pedido
actor "Cliente" as Cliente
actor "Vendedor" as Vendedor
boundary "PantallaPedido" as UI
boundary "AlertaPedido" as Alerta
control "ControlPedido" as Ctrl
entity "Pedido" as Ped
entity "Producto" as Prod
database "RepositorioProducto" as RepoProd
database "RepositorioPedido" as RepoPed

Cliente -> UI : Selecciona productos, cantidades y ubicación
```

```
Cliente -> UI : Clic en Realizar pedido
UI -> Ctrl : realizarPedido(idCliente, items, ubicacion)
Ctrl -> Ctrl : validarStock(items)
loop Por cada producto del pedido
  Ctrl -> RepoProd : buscarPorId(idProducto)
  RepoProd --> Ctrl : Producto encontrado
  Ctrl -> Prod : hayStock(cantidad)
  Prod --> Ctrl : true
  Ctrl -> RepoProd : reservarStock(idProducto, cantidad)
end
Ctrl -> Ped : new Pedido(idCliente, items, ubicacion, "pendiente")
Ctrl -> RepoPed : guardar(Ped)
RepoPed --> Ctrl : guardado correcto
Ctrl -> Ctrl : notificarPedido(Ped)
Ctrl -> Alerta : mostrarAlerta(Ped)
Alerta --> Vendedor : Alerta visual y sonora de nuevo pedido
Ctrl --> UI : Resultado exitoso
UI --> Cliente : Muestra pedido pendiente
@enduml
```

8.3 Secuencia: Realizar Pago (incluye Pagar Efectivo y Pagar Transferencia)

```
@startuml
title Secuencia de análisis - Realizar Pago
actor "Cliente" as Cliente
actor "Vendedor" as Vendedor
actor "Banca Móvil" as Banca
boundary "PantallaPago" as UI
boundary "InterfazBancaMovil" as IBM
control "ControlPago" as Ctrl
entity "Pago" as Pag
database "RepositorioPago" as RepoPag

Cliente -> UI : Selecciona pedido o reserva y método de pago
Cliente -> UI : Clic en Realizar pago
UI -> Ctrl : realizarPago(idReferencia, metodo, monto)
Ctrl -> Pag : new Pago(idReferencia, metodo, monto, "pendiente")
Ctrl -> RepoPag : guardar(Pag)
RepoPag --> Ctrl : guardado correcto

alt Método = efectivo (Pagar Efectivo)
  Vendedor -> UI : Registra el monto recibido
  Vendedor -> UI : Clic en Pagar efectivo
  UI -> Ctrl : pagarEfectivo(idPago, montoRecibido)
  Ctrl -> RepoPag : buscarPorId(idPago)
```

```
  RepoPag --> Ctrl : Pago encontrado
  Ctrl -> Ctrl : validarMonto(monto, montoRecibido)
  Ctrl -> Pag : confirmar()
else Método = transferencia (Pagar Transferencia)
  Cliente -> UI : Sube o envía el comprobante
  UI -> Ctrl : pagarTransferencia(idPago, comprobante)
  Ctrl -> RepoPag : buscarPorId(idPago)
  RepoPag --> Ctrl : Pago encontrado
  Ctrl -> IBM : solicitarConfirmacion(idPago, Pag.getMonto(), comprobante)
  IBM -> Banca : Consulta la transferencia
  Banca --> IBM : Confirma o rechaza transacción
  IBM --> Ctrl : boolean (true / false)
  alt Respuesta = true (Confirmada)
    Ctrl -> Pag : confirmar()
  else Respuesta = false (Rechazada)
    Ctrl -> Pag : rechazar()
  end
end

Ctrl -> RepoPag : actualizar(Pag)
RepoPag --> Ctrl : actualización correcta
Ctrl --> UI : Resultado del pago
UI --> Cliente : Muestra el estado del pago
@enduml
```

8.4 Secuencia: Generar Reporte de Ingresos

```
@startuml
title Secuencia de análisis - Generar Reporte de Ingresos
actor "Administrador" as Admin
boundary "PantallaReporteIngresos" as UI
control "ControlReporte" as Ctrl
database "RepositorioPago" as RepoPag

Admin -> UI : Ingresa fechas y método de pago
Admin -> UI : Clic en Generar reporte
UI -> Ctrl : generarReporteIngresos(fechaInicio, fechaFin, metodoPago)
Ctrl -> RepoPag : listarConfirmados(fechaInicio, fechaFin, metodoPago)
RepoPag --> Ctrl : listaPagos
Ctrl -> Ctrl : calcularTotal(listaPagos)
Ctrl --> UI : listaPagos y total
UI --> Admin : Muestra el reporte de ingresos
@enduml
```

9. Especificación breve de casos de uso conectada con las secuencias

Caso de uso	Precondición	Flujo principal resumido	Resultado esperado
Realizar Reserva (GRC-27_1)	La cancha debe estar habilitada y disponible en el horario solicitado.	Ingresar cancha, fecha y horario → validar disponibilidad → crear reserva pendiente.	Reserva guardada en estado «pendiente» y enviada al administrador.
Aceptar Reserva (GRC-27_2)	El cliente debe haber realizado la reserva.	Seleccionar reserva pendiente → buscar → confirmar → actualizar.	Reserva confirmada y cancha bloqueada en el horario reservado.
Realizar Pedido (GT-03)	El cliente debe estar registrado y existir productos con stock en el catálogo.	Seleccionar productos y ubicación → validar stock → reservar stock → guardar pedido pendiente.	Pedido registrado en estado «pendiente» y notificado al vendedor.
Notificar Pedido (GT-04)	Debe haberse registrado un pedido.	Detectar pedido nuevo → generar notificación → mostrar alerta al vendedor.	Vendedor informado del nuevo pedido para su despacho.
Realizar Pago (GI-33)	Debe existir un pedido o una reserva pendiente de pago.	Elegir método de pago → crear pago pendiente → derivar a efectivo o transferencia.	Pago registrado y actualizado según el método elegido.
Pagar Efectivo (GI-34)	El cliente paga de forma presencial.	Registrar monto recibido → validar contra el valor a pagar → confirmar el pago.	Pago en efectivo confirmado y asociado al pedido o reserva.
Pagar Transferencia (GI-35)	El cliente debe contar con banca móvil habilitada.	Subir comprobante → consultar a la banca móvil → confirmar o rechazar → actualizar el pago.	Pago «confirmado» o «rechazado» según la respuesta de la banca.
Generar Reporte de Ingresos (GI-32)	Debe existir al menos un pago registrado.	Ingresar filtros → listar pagos confirmados → calcular total → mostrar reporte.	Reporte de ingresos del periodo con el total calculado.

10. Procedimiento recomendado para el estudiante

1. Revisar el SRS y el documento de casos de uso del SPR, e identificar los datos que maneja el sistema: reserva, pedido, pago y reporte.
2. Redactar los requisitos funcionales de nivel 0: Realizar Reserva, Realizar Pedido, Realizar Pago y Generar Reporte de Ingresos, más los requisitos de apoyo.
3. Modelar los casos de uso en PlantUML y justificar únicamente las relaciones `<<extend>>` necesarias.
4. Derivar las clases de análisis: una clase frontera por pantalla, una clase control por caso de uso principal, las entidades del dominio y un repositorio por entidad.
5. Construir un diagrama de secuencia por cada caso de uso principal.
6. Verificar que cada mensaje del diagrama de secuencia corresponda a una operación declarada en el diagrama de clases.
7. Actualizar la matriz de relación para demostrar coherencia entre RF, CU, clases y secuencias.

11. Lista de verificación de coherencia

- Cada requisito funcional tiene al menos un caso de uso asociado.
- Cada caso de uso principal posee una secuencia de análisis propia.
- Las clases usadas en las secuencias existen en el diagrama de clases de análisis.
- Los mensajes de secuencia coinciden con operaciones de las clases de análisis.
- Las entidades Reserva, Cancha, Pedido, Producto y Pago conservan los datos definidos en el caso base.
- La validación de disponibilidad, de stock y de monto se aplica antes de guardar o confirmar.
- El diagrama no mezcla detalles de implementación de interfaz gráfica con responsabilidades de análisis.

12. Rúbrica breve de evaluación sobre 20 puntos

Criterio	Descripción	Puntaje	Evidencia
Trazabilidad entre modelos	Relaciona correctamente RF, casos de uso, clases y secuencias.	5 pts	Matriz de relación completa.
Diagrama de casos de uso	Representa actor, sistema, casos de uso y relaciones include justificadas.	4 pts	Código y diagrama PlantUML.
Diagrama de clases de análisis	Identifica frontera, control, entidad y repositorio con responsabilidades claras.	4 pts	Código PlantUML de clases.

Diagramas de secuencia	Modela flujos coherentes para Realizar Reserva, Realizar Pedido, Realizar Pago y Generar Reporte de Ingresos	5 pts	Cuatro secuencias PlantUML.
Presentación técnica	Documento ordenado, lenguaje claro y sin errores de sintaxis PlantUML.	2 pts	Entrega final documentada.

13. Formato mínimo de entrega sugerido

Sección	Contenido mínimo	Evidencia
Portada	Datos institucionales, asignatura, NRC, estudiante/equipo, proyecto y fecha.	Documento entregado.
Casos de uso	Requisitos funcionales, diagrama CU y especificación breve.	Código PlantUML + captura.
Clases de análisis	Diagrama con boundary, control, entity y repositorio.	Código PlantUML + captura.
Secuencias de análisis	Una secuencia por cada operación CRUD principal.	Códigos PlantUML + capturas.
Trazabilidad	Matriz que conecte RF, CU, clases y secuencias.	Tabla de relación.
Conclusión técnica	Reflexión breve sobre la coherencia entre modelos.	Texto de cierre.

14. Conclusión técnica

La relación entre casos de uso, clases de análisis y diagramas de secuencia permite comprobar que el modelo del SPR no se construye como piezas aisladas. Los casos de uso explican qué necesita cada actor: el cliente reserva, pide y paga; el vendedor atiende las alertas y cobra en efectivo; el administrador acepta reservas y consulta los ingresos; y la banca móvil confirma las transferencias. Las clases de análisis explican qué objetos colaboran para lograrlo: pantallas, controles, entidades del dominio y repositorios. Las secuencias muestran cómo esos objetos interactúan en el tiempo para cumplir cada caso de uso, incluidas las validaciones críticas del SRS, como la ausencia de solapamiento en las reservas, el stock suficiente en los pedidos y la confirmación de la banca móvil en las transferencias.

Cada mensaje de las secuencias corresponde a una operación declarada en el diagrama de clases, y cada caso de uso mantiene su vínculo con un requisito funcional del SRS. Esta trazabilidad mejora la claridad del análisis, reduce el riesgo de inconsistencias entre los modelos y deja preparado el diseño posterior del sistema.

15. Fuente interna de referencia

- Guía 1: diagramas de casos de uso de análisis del SPR (versión final).

- Especificación de Requisitos de Software (SRS) del SPR – Canchas Don Pepito, versión 4.0, julio 2026.
- Documento integrado de casos de uso: Propuesta de Solución 1 y Propuesta de Solución 2 (módulo de Gestión de Ingresos), 01/07/2026.
- Guía 2 proporcionada por la docente, adaptada al proyecto SPR.